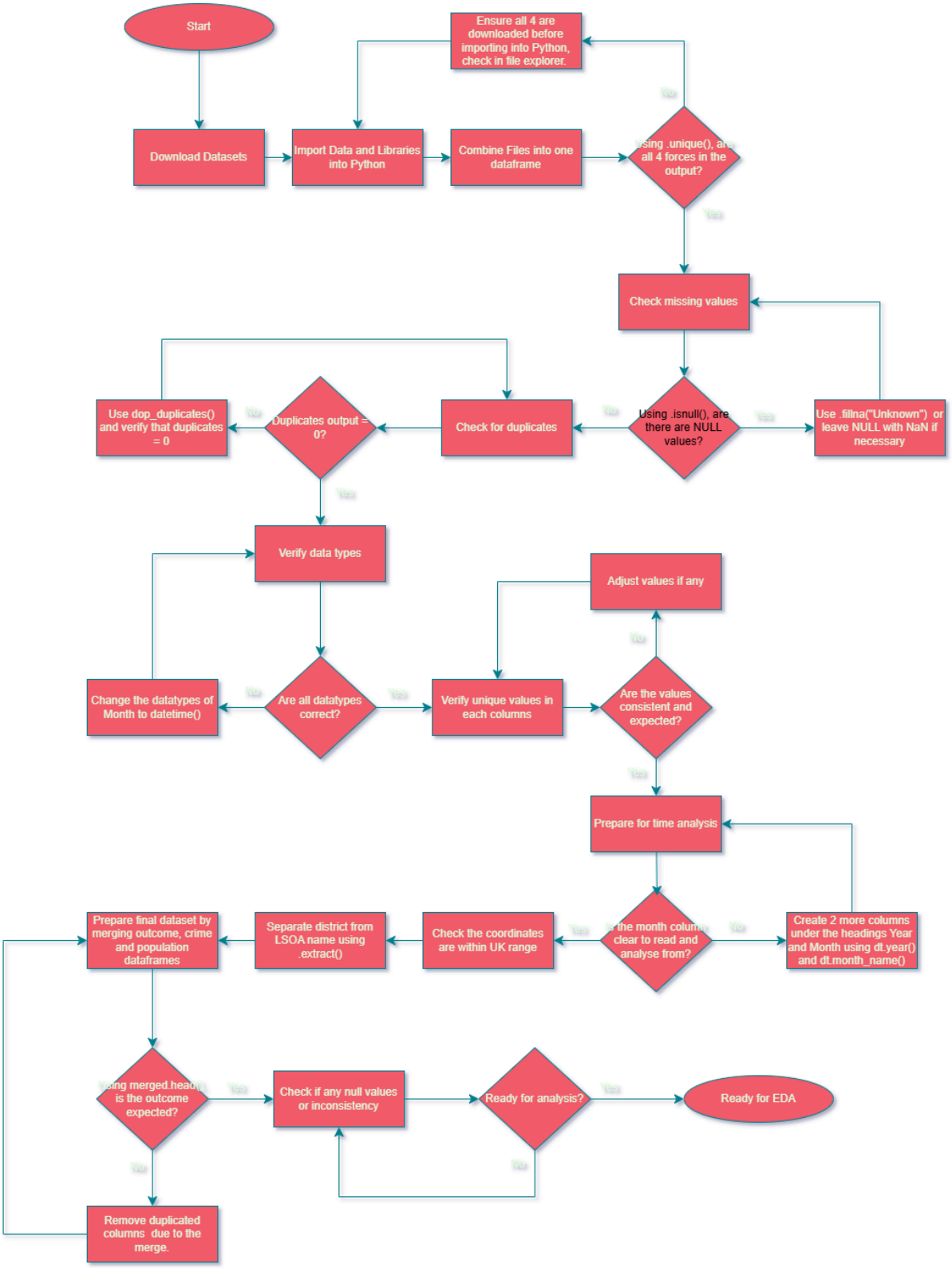
Data Preprocessing

The steps below correlate to the flowchart steps, with each step having a detailed description of why and how.



Download Datasets / Data Collection

The open-source datasets come from Police Data with having Metropolitan Police Service, Devon & Cornwall Police, North Yorkshire Police and West Midlands Police as the selected police force. By selecting areas that are in different regions, this allows for the analysis between rural and urban areas. The outcome data was also from here.

Link to Police data: https://data.police.uk/data/

The population dataset was researched and came from another open-source from a LSOA Population estimates website, where mid 2024 totals and codes were extracted to use for the current dataset.

Link to LSOA Population estimates:
https://www.ons.gov.uk/peoplepopulationandcommunity/populationandmigration/populationestimates/datasets/lowersuperoutputareamidyearpopulationestimatesnationalstatistics

Import Data and Libraries into Python

Importing libraries allow the necessary tools to be used for data manipulation, analysis and preprocessing. In this project, libraries such as *panda, numpy, matplotlib* and *seaborn* were imported to handle data structures and numerical operations efficiently.

The dataset was then imported into Python using .read_csv() from pandas, converting CSV files into DataFrame objects.

Combine Files into one dataframe

Using the .concat() function, allows for all the separate files to be placed into one DataFrame. This makes it easier to manage and analyse the data in one unified structure, ensuring that all records from the different files are included for further preprocessing and analysis.

Using .unique(), are all 4 forces in the output?

By using .unique(), the distinct values within a specific column can be identified. This allows verification that all four police forces are present in the dataset after combining the files.

- ***If NO: Ensure all 4 are downloaded before importing into Python, check in file explorer.***

  Checking in the file explorer that all four police forces is the most convenient way to ensure all files are there, if not, the file was redownloaded from the website.

Check missing values

Checking for missing values ensures that the dataset does not contain incomplete or undefined entries that could affect analysis. This step helps identify NULL or NaN values within the dataset so they can be handled appropriately during preprocessing.

Using .isnull(), are there NULL values?

Using .isnull().sum() allowed an insight to see how many nulls per column.

```
Crime ID                  616904
Month                          0
Reported by                    0
Falls within                   0
Longitude                  17169
Latitude                   17169
Location                       0
LSOA code                  17177
LSOA name                  17177
Crime type                     0
Last outcome category     616904
Context                  3493686
dtype: int64
```

- ***IF YES: Use .fillna("Unknown") or leave NULL with NaN if necessary***

- 'Crime ID was left unchanged as it functions as a unique identifier and is required to link the street crime data to the outcomes dataset. Missing values were preserved to avoid introducing later errors.

- Longitude and Latitude left unchanged as missing values in the columns were retained as NaN to avoid introducing inaccurate geographic coordinates of 0. Changing these to 'Unknown' would also change the column to the incorrect datatype, by leaving NaN, the datatype stays correctly as float rather than object.

- The column "Last outcome category" contained a large number of missing values and was removed from the dataset. Since a separate outcomes dataset was obtained from the UK Police Data platform, this column was considered redundant for the analysis.

- For every other column, NaN was replaced with Unknown

Check for duplicates

Checking for duplicates ensures that no identical records appear more than once in the dataset. This helps maintain data accuracy and prevents duplicated entries from skewing analysis or statistical results.

Duplicates output = 0?

```
np.int64(283263)
```

The image above shows how many duplicates there were before cleaning.

- ***Use drop_duplicates() and verify that duplicates = 0***
-
```
np.int64(0)
```
- After dropping the duplicates

Verify data types

Verifying data types ensures that each column in the dataset is stored in an appropriate format for analysis. This step helps confirm that numerical, categorical, and date values are correctly recognised, allowing functions and calculations to be applied accurately.

Are all datatypes correct?

```
Crime ID         object
Month            object
Reported by      object
Falls within     object
Longitude       float64
Latitude        float64
Location         object
LSOA code        object
LSOA name        object
Crime type       object
dtype: object
```

The image above shows the data types for each object and Month shows that it is still an object.

- ***Change the datatypes of Month to datetime()***
-
```
Crime ID                 object
Month            datetime64[ns]
Reported by              object
Falls within             object
Longitude               float64
Latitude                float64
Location                 object
LSOA code                object
LSOA name                object
Crime type               object
dtype: object
```

- After converting the datatype using pd.to_datetime(), time series analysis can be done later on in the EDA stage.

Verify unique values in each columns

This step involves verifying that the values within each column are consistent and fall within the expected range or format. Identifying unexpected or inconsistent values helps ensure the dataset is accurate and reliable before proceeding with further analysis.

Prepare for time analysis

Preparing the dataset for time analysis involves ensuring that the date or time-related columns are in the correct format for temporal analysis.

Create 2 more columns under the headings Year and Month using dt.year() and dt.month_name()

Year	Month
2024	January
2024	January
2024	January
2024	January
2024	January

Using dt.month_name() and dt.year(), allowed the separation of the Month column into two separate Year Month columns to allow for analysis between years and months separately. This allows trends, patterns, and changes over time to be accurately examined during the exploratory data analysis stage.

Check the coordinates are within UK range

The UK coordinates usually fall between:

Longitude = -8 - 2
Latitude = 49 - 61

This step verifies that the geographic coordinates fall within the expected range for locations within the United Kingdom.

Separate district from LSOA name using .extract()

This step separates the district name from the LSOA field to make the geographic information clearer and easier to analyse. By using the .extract() function, the relevant part of the text can be isolated and stored in a new column, allowing district-level analysis to be performed more effectively.

Prepare final dataset by merging outcome, crime and population dataframes

The final dataset was prepared by merging the outcome, crime, and population DataFrames into a single unified dataset. This step allows information from each source to be combined, enabling more comprehensive analysis by linking crime records with their corresponding outcomes and population data.

Using .head(), is the outcome expected?

The .head() function was used to inspect the first few rows of the merged dataset to confirm that the merge operation produced the expected result. During this check, it was observed that multiple columns were duplicated because both DataFrames contained similar fields after the merge.

- ***Remove duplicated columns due to the merge***
- Using suffixes within the pandas joining function, allows to change:
    - Example: LSOA_X and LSOA_Y to LSOA_crime and LSOA_outcome. This is to differentiate the different but similar columns. As both columns have the same values and rows, the outcome duplicates were dropped.

Check if any null values or inconsistency

Ready for analysis?

Checking through the merged DataFrame using learnt functions and ensuring the DataFrame is ready for analysis to prevent having to go back and forth editing/updating.

Ready for EDA

Ready for the EDA stage.